Password Strength + Signup Form

Problem: Build a signup form (name, email, password, confirm password, terms) with a live password strength meter and a rule checklist driven by regex rules.

Tech stack: React (Vite, JavaScript) (default, none given)

Time limit: 60 minutes (default, none given)

Often asked at: Paytm, PhonePe

PART A: Thought process

Steps that work for any machine coding task:

1. **Clarify** the requirements and write down the assumptions.
2. **List the interactions** (action -> expected result). This becomes your checklist.
3. **Sketch the components** and who owns which state.
4. **Design the state and data** (what is stored, what is derived).
5. **Find the 2-4 hard parts** before coding so they do not surprise you at minute 50.
6. **Build in layers:** a working skeleton first, then one feature at a time, always runnable.
7. **Handle states and edge cases** (empty, loading, error, success).
8. **Wrap up:** walk the interviewer through trade-offs and what you would add.

Time budget for 60 minutes

Minutes	Activity
0-7	Requirements, clarifying questions, assumptions
7-15	Interactions, components, state, rule data shape
15-45	Build: fields -> rules + checklist -> meter -> validation -> submit
45-53	States, edge cases, accessibility pass
53-60	Demo, trade-offs, what to add next

Think out loud. Say why you picked a rule list over one big regex, why errors wait for blur, why strength is derived. The interviewer grades your reasoning as much as the result.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Fields: full name, email, password, confirm password, accept terms checkbox.
- As the user types a password, show a **checklist of rules** (8+ chars, lowercase, uppercase, number, special char, no personal info) that tick live.
- Show a **strength meter** (Very weak -> Strong) with colored bars and a text label.
- Show/hide password toggle.
- Validate every field; block submit until the form is valid.
- Confirm password must match.
- Submit to a (mock) API, show loading, then success or a server error.

Non-functional requirements

Area	What it means here
Performance	Strength is computed on every keystroke; must be O(length), no lag.
Scalability	Adding a rule = adding one object to a list, no UI change.
Error handling	Field errors, server errors ("email already exists"), no double submit.
Accessibility	Labels, `aria-invalid`, `aria-describedby`, live region for strength, not color-only.
Reusability	`PasswordInput`, `StrengthMeter`, `RuleChecklist` usable in a "change password" page too.
Extensibility	Rules and scoring live in a pure module; can be shared with the backend.

Clarifying questions

1. What exact password policy? (min length, which character classes, max length?)
2. Should a weak-but-valid password be allowed, or must all rules pass?
3. When should errors show: on every keystroke, on blur, or only on submit?
4. Is there a real API? What does it return for an email that already exists?
5. Should we block common passwords (`Password@123`) and passwords containing the user's name/email?
6. Any i18n or accessibility (WCAG) requirements?

Assumptions we build with

- Policy: 8+ chars, 1 lowercase, 1 uppercase, 1 digit, 1 special char, must not contain name or email.
- Submit allowed only when **all rules pass** (score >= Good) and the password is not on a common list.
- Errors appear after a field is **blurred** or after the first submit; the checklist is always live.
- A mock API with 800 ms latency; `taken@example.com` already exists.
- Password is never trimmed (spaces are legal characters); name and email are trimmed.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Empty form, checklist with all rules unticked, no meter, no errors
Type in password	Checklist items tick live; meter appears with bars + label
Type `Password@123`	All regex rules pass, but label says **Too common**, score 0
Type password containing your name	"Does not contain your name or email" stays unticked

Action / input	Expected behavior / output
Password with all rules, 8-11 chars	**Good** (4 bars)
Password with all rules, 12+ chars	**Strong** (5 bars)
Blur a field with bad input	Field error shows under it, input gets red border
Confirm differs from password	"Passwords do not match" (after confirm is touched)
Change password after confirm matched	Mismatch error appears again (derived, never stale)
Click Show / Hide	Input switches `password` <-> `text`, `aria-pressed` updates
Caps Lock on while typing password	"Caps Lock is on" hint
Submit with errors	All errors shown, focus moves to the first invalid field
Submit valid form	Button disabled, text "Creating account..."
Click submit again while submitting	Ignored (no double request)
API says email exists	Red alert banner with server message, form stays filled
Edit any field after server error	Banner clears
API success	Welcome screen with name and email; "Sign up another" resets

3. Design the architecture

Module	Responsibility
`utils/passwordRules.js`	Pure: `RULES`, `getStrength()`, `validate()`, common list. No React.
`services/authApi.js`	Mock `signup()` returning a Promise (latency + duplicate email).
`App.jsx`	Switches between the form and the success screen.
`SignupForm.jsx`	Owns form state, touched, submit status; wires everything.
`TextField.jsx`	Label + input + error with correct ARIA.
`PasswordInput.jsx`	Same, plus show/hide toggle and Caps Lock hint.
`StrengthMeter.jsx`	Five bars + live text label.
`RuleChecklist.jsx`	List of rules with met / not met state.

```
App
 ├── SignupForm               (values, touched, status, serverError)
 │    ├── TextField (name)
 │    ├── TextField (email)
 │    ├── PasswordInput (password)   (visible, capsLock: local UI state)
 │    ├── StrengthMeter              (props: score, label)
 │    ├── RuleChecklist              (props: rules[])
 │    ├── PasswordInput (confirm)
 │    └── checkbox (terms) + submit
```

```
  └── Success screen            (user)

SignupForm ──uses──> passwordRules.js (pure)  and  authApi.js (async)
```

Start with everything inside `SignupForm`, then pull out `PasswordInput` once you need the toggle twice (password + confirm), and pull the rules into a pure file as soon as both the checklist and `validate` need them.

4. Design the state / data model

State or field	Purpose
`values` `{ name, email, password, confirm, terms }`	Controlled input values (single object, one change handler)
`touched` `{ [field]: true }`	Which fields were blurred: decides when to show an error
`status` `'idle' | 'submitting' | 'error'`	Drives button disabled/text and double-submit guard
`serverError`	Message from the API, shown as an alert
`visible` (in `PasswordInput`)	Show / hide password, local UI state
`capsLock` (in `PasswordInput`)	Caps Lock hint, local UI state
`user` (in `App`)	Created user; non-null shows the success screen

```
const INITIAL = { name: '', email: '', password: '', confirm: '', terms: false }

const [values, setValues] = useState(INITIAL)
const [touched, setTouched] = useState({})
const [status, setStatus] = useState('idle')
const [serverError, setServerError] = useState('')

// derived, NOT state
const strength = getStrength(values.password, values)
const errors = validate(values)
```

Senior note

- `strength`, `rules[].passed`, `errors` and "passwords match" are **derived** from `values`. Storing them would mean syncing them in every handler, and the classic bug is: confirm matched, user edits password, the stored "match" flag is now wrong. Deriving makes that bug impossible.
- **Ownership:** form data lives in `SignupForm` (the only place that submits it). Show/hide and Caps Lock live in `PasswordInput` because nobody else cares. `user` lives in `App` because it decides which screen to render.
- `isValid` is just `Object.keys(errors).length === 0`.

5. Design the data structures and contracts

Rules as data (stable `id` used as React key and in tests):

```
export const RULES = [
  { id: 'length',  label: 'At least 8 characters', test: (p) => p.length >= 8 },
  { id: 'lower',   label: 'One lowercase letter',  test: (p) => /[a-z]/.test(p) },
  { id: 'upper',   label: 'One uppercase letter',  test: (p) => /[A-Z]/.test(p) },
  { id: 'digit',   label: 'One number',            test: (p) => /\d/.test(p) },
  { id: 'special', label: 'One special character', test: (p) => /[^A-Za-z0-9\s]/.test(p) },
  { id: 'personal', label: 'Does not contain your name or email',
    test: (p, ctx) => !personalTokens(ctx).some((t) => p.toLowerCase().includes(t)) },
]
```

Why: one list feeds the checklist UI, the score and `validate()`. A new rule is one line, and the same file could be shared with a Node backend.

`getStrength(password, ctx)` contract

```
getStrength('Tr0ub4dor&x', { name: 'Ashu', email: 'a@b.co' })
// -> {
//   rules: [{ id: 'length', label: '...', passed: true }, ...],
//   score: 3,            // 0..4
//   label: 'Good',       // 'Very weak' | 'Weak' | 'Fair' | 'Good' | 'Strong' | 'Too common' | ''
//   common: false
// }
```

`validate(values)` contract: returns `{ [field]: message }`, empty object when valid. Key order matches DOM order so `Object.keys(errors)[0]` is the first invalid field.

API contract (mocked)

```
POST /api/signup
Request : { "name": "Ashu Kumar", "email": "ashu@example.com", "password": "Tr0ub4dor&xyz9" }
201     : { "id": "6f1c...", "name": "Ashu Kumar", "email": "ashu@example.com" }
409     : { "message": "An account with this email already exists" }
```

```
export function signup({ name, email, password }) // Promise<{ id, name, email }>, rejects with
Error(message)
```

6. Trace the data flow

Key interaction: user types one more character into the password field.

```
keystroke "9" in #password
   │
   ▼
onChange -> handleChange(e) -> setValues({ ...v, password: 'Tr0ub4dor&xyz9' })
   │                           (serverError cleared if present)
   ▼
SignupForm re-renders
   │
   ├── getStrength(password, values)
   │      ├── run each RULES[i].test(password, ctx)   -> rules[].passed
   │      ├── count passed, length bonus               -> score 4
   │      └── common-list check                        -> label 'Strong'
   │
   └── validate(values)  -> errors (password ok, confirm now mismatched?)
   │
   ▼
Output
   ├── RuleChecklist ticks the rules that passed
   ├── StrengthMeter fills 5 bars green, aria-live announces "Strength: Strong"
   └── confirm error appears only if confirm was touched (touched.confirm)
```

7. Important technical challenges

Challenge 1: Rules as regex, one source of truth

Problem: A single giant regex like `/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s]).{8,}$/` tells you *pass/fail* but not *which rule failed*, so the checklist cannot tick items individually, and messages drift from the regex.

Solution: One small regex per rule inside a data list. The checklist maps over the results; `validate` and the score read the same results.

```
const rules = RULES.map((r) => ({ id: r.id, label: r.label, passed: password !== '' && r.test(password, ctx) }))
```

Note `password !== ''`: without it, "Does not contain your name" would show ticked on an empty field.

Challenge 2: A score that regex alone cannot fool

Problem: `Password@123` passes every character-class regex but is one of the most leaked passwords. `Kumar#2026xyz` from user "Ashu Kumar" is guessable.

Solution: Bucket the number of passed rules into 0-4, give the top score only to long passwords, and override to 0 for a common-list hit. The "personal" rule uses name and email tokens from the same `values`.

```
const passed = rules.filter((r) => r.passed).length
const allPassed = passed === RULES.length
let score = allPassed ? (password.length >= 12 ? 4 : 3) : Math.floor(passed / 2)
```

```
if (common) score = 0
return { rules, score, label: common ? 'Too common' : LEVELS[score], common }
```

Challenge 3: When to show errors (and never show stale ones)

Problem: Showing "Email is required" while the user has not even reached the email field is hostile. But hiding errors until submit means they discover 4 errors at once. And a stored "passwords match" flag goes stale when the password changes.

Solution: Errors are always computed (derived) but only **shown** for fields in `touched`. Blur marks a field touched; submit marks all of them touched and focuses the first invalid one.

```
const errors = validate(values)
const errorFor = (field) => (touched[field] ? errors[field] : undefined)

// on submit
setTouched(ALL_TOUCHED)
const firstInvalid = Object.keys(errors)[0]
if (firstInvalid) return e.currentTarget.elements[firstInvalid].focus()
```

Challenge 4: Accessible meter, toggle and async submit

Problem: Color-only bars mean nothing to a screen reader or a color-blind user; a show/hide icon without a label is unusable; a double click sends two signup requests.

Solution: Text label in an `aria-live="polite"` region, bars `aria-hidden`, checklist items with "(met)/(not met)" sr-only text, input linked to meter and checklist via `aria-describedby`. The toggle is a real `<button type="button">` with `aria-pressed`. Submit is guarded by `status`.

```
<p className={`strength-label text-${score}`} aria-live="polite">Strength: <strong>{label}</strong></p>

<button type="button" aria-pressed={visible} aria-label={visible ? 'Hide password' : 'Show password'}
  onClick={() => setVisible((v) => !v)}>{visible ? 'Hide' : 'Show'}</button>

if (submitting) return            // double-submit guard
<button type="submit" disabled={submitting}>...</button>
```

8. Implement in the correct order

#	Step	Working result
1	Static form: 5 fields + submit, controlled `values`, one `handleChange`	Can type everywhere
2	`RULES` list + checklist mapped from it	Rules tick live
3	`getStrength` scoring + `StrengthMeter` bars and label	Meter moves with typing
4	Common-password and personal-info checks	`Password@123` shows Too common

#	Step	Working result
5	`validate(values)` for all fields, including confirm match	Errors object correct
6	`touched` + blur, show errors only for touched fields	Friendly error timing
7	Submit: touch all, focus first invalid, block if errors	Can't submit invalid form
8	Mock `signup()` + `status` (submitting, error) + double-submit guard	Loading and server error
9	Success screen in `App` + reset	Full happy path
10	Extract `TextField`, `PasswordInput` with show/hide + Caps Lock	Reusable pieces
11	Accessibility pass: labels, `aria-*`, live region, focus styles	Screen reader friendly
12	CSS polish + quick self-check of `getStrength` / `validate`	Done

9. Cover all states and edge cases

State / case	What the UI shows
Initial	Empty fields, checklist all unticked, no meter, no errors
Typing password	Meter + label, checklist updates every keystroke
Empty password after typing	Meter disappears, checklist resets (no ticks on empty)
Common password	Score 0, label "Too common", submit error "too common"
Contains name / email	Personal rule unticked, score capped below Good
All rules, short	Good (4 bars)
All rules, 12+ chars	Strong (5 bars)
Invalid email (`a@b`)	"Enter a valid email address" after blur
Whitespace-only name	"Name is required" (trimmed)
Confirm mismatch, then password edited	Error recomputed from current values, never stale
Caps Lock on	"Caps Lock is on" hint under the field
Submit with errors	All errors visible, focus on first invalid field
Submitting (loading)	Button disabled, "Creating account...", `aria-busy` on form
Double click submit	Second click ignored
Server error (email exists)	`role="alert"` banner, values kept
Edit after server error	Banner cleared
Success	Welcome screen with name + email, "Sign up another" resets
Paste into password	Same as typing (handled by `onChange`)
Browser password manager	`autoComplete="new-password"` lets it suggest a strong one

10. Final structure

```
_89_password_strength_signup/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── App.jsx                    # form vs success screen
    ├── index.css
    ├── components/
    │   ├── SignupForm.jsx         # values, touched, status, submit
    │   ├── TextField.jsx          # label + input + error
    │   ├── PasswordInput.jsx      # + show/hide, Caps Lock hint
    │   ├── StrengthMeter.jsx      # 5 bars + aria-live label
    │   └── RuleChecklist.jsx      # met / not met list
    ├── services/
    │   └── authApi.js             # mock signup()
    └── utils/
        └── passwordRules.js       # RULES, getStrength, validate (pure)
```

11. Full working code

src/utils/passwordRules.js

```
// Every rule is data: one list drives the checklist, the meter and the validation.
const personalTokens = ({ name = '', email = '' }) =>
  [...name.toLowerCase().split(/\s+/), email.toLowerCase().split('@')[0]].filter((t) => t.length >= 3)

export const RULES = [
  { id: 'length', label: 'At least 8 characters', test: (p) => p.length >= 8 },
  { id: 'lower', label: 'One lowercase letter', test: (p) => /[a-z]/.test(p) },
  { id: 'upper', label: 'One uppercase letter', test: (p) => /[A-Z]/.test(p) },
  { id: 'digit', label: 'One number', test: (p) => /\d/.test(p) },
  { id: 'special', label: 'One special character (!@#$...)', test: (p) => /[^A-Za-z0-9\s]/.test(p) },
  {
    id: 'personal',
    label: 'Does not contain your name or email',
    test: (p, ctx) => !personalTokens(ctx).some((t) => p.toLowerCase().includes(t)),
  },
]

// shortcut: tiny in-memory list, swap for a server-side breached-password check (e.g. HIBP k-
anonymity) in production
const COMMON = new Set([
  'password', 'password1', 'password123', 'password@123', 'password1!', 'passw0rd',
```

```
  '12345678', '123456789', '1234567890', 'qwerty123', 'iloveyou', 'admin123',
  'welcome1', 'welcome@123', 'letmein1', 'abc12345', 'india@123',
])

export const LEVELS = ['Very weak', 'Weak', 'Fair', 'Good', 'Strong']

export function getStrength(password, ctx = {}) {
  const rules = RULES.map((r) => ({ id: r.id, label: r.label, passed: password !== '' &&
r.test(password, ctx) }))
  if (!password) return { rules, score: 0, label: '', common: false }

  const common = COMMON.has(password.toLowerCase())
  const passed = rules.filter((r) => r.passed).length
  const allPassed = passed === RULES.length
  // 0-1 rules -> 0, 2-3 -> 1, 4-5 -> 2, all -> 3, all + 12 chars -> 4
  let score = allPassed ? (password.length >= 12 ? 4 : 3) : Math.floor(passed / 2)
  if (common) score = 0

  return { rules, score, label: common ? 'Too common' : LEVELS[score], common }
}

export const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/

export function validate(values) {
  const errors = {}
  const email = values.email.trim()
  if (!values.name.trim()) errors.name = 'Name is required'

  if (!email) errors.email = 'Email is required'
  else if (!EMAIL_RE.test(email)) errors.email = 'Enter a valid email address'

  const strength = getStrength(values.password, values)
  if (!values.password) errors.password = 'Password is required'
  else if (strength.common) errors.password = 'This password is too common, pick another'
  else if (strength.score < 3) errors.password = 'Password does not meet all requirements'

  if (!values.confirm) errors.confirm = 'Please confirm your password'
  else if (values.confirm !== values.password) errors.confirm = 'Passwords do not match'

  if (!values.terms) errors.terms = 'You must accept the terms'
  return errors
}
```

src/services/authApi.js

```
// Mock backend: 800ms latency, one email is already registered.
const TAKEN = new Set(['taken@example.com'])
```

```
export function signup({ name, email, password }) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const key = email.toLowerCase()
      if (TAKEN.has(key)) return reject(new Error('An account with this email already exists'))
      if (password.length < 8) return reject(new Error('Password rejected by server'))
      TAKEN.add(key)
      resolve({ id: crypto.randomUUID(), name, email: key })
    }, 800)
  })
}
```

src/components/SignupForm.jsx

```
import { useState } from 'react'
import TextField from './TextField'
import PasswordInput from './PasswordInput'
import StrengthMeter from './StrengthMeter'
import RuleChecklist from './RuleChecklist'
import { getStrength, validate } from '../utils/passwordRules'
import { signup } from '../services/authApi'

const INITIAL = { name: '', email: '', password: '', confirm: '', terms: false }
const ALL_TOUCHED = { name: true, email: true, password: true, confirm: true, terms: true }

export default function SignupForm({ onSuccess }) {
  const [values, setValues] = useState(INITIAL)
  const [touched, setTouched] = useState({})
  const [status, setStatus] = useState('idle') // idle | submitting | error
  const [serverError, setServerError] = useState('')

  // Derived every render: no stale strength or stale "passwords do not match".
  const strength = getStrength(values.password, values)
  const errors = validate(values)
  const errorFor = (field) => (touched[field] ? errors[field] : undefined)
  const submitting = status === 'submitting'

  const handleChange = (e) => {
    const { name, type, value, checked } = e.target
    setValues((v) => ({ ...v, [name]: type === 'checkbox' ? checked : value }))
    if (serverError) setServerError('')
  }

  const handleBlur = (e) => setTouched((t) => ({ ...t, [e.target.name]: true }))

  const handleSubmit = async (e) => {
```

```
    e.preventDefault()
    if (submitting) return
    setTouched(ALL_TOUCHED)

    const firstInvalid = Object.keys(errors)[0]
    if (firstInvalid) {
      e.currentTarget.elements[firstInvalid].focus()
      return
    }

    setStatus('submitting')
    try {
      const user = await signup({ name: values.name.trim(), email: values.email.trim(), password:
values.password })
      onSuccess(user)
    } catch (err) {
      setServerError(err.message)
      setStatus('error')
    }
  }

  return (
    <form onSubmit={handleSubmit} noValidate aria-busy={submitting}>
      {serverError && <p role="alert" className="server-error">{serverError}</p>}

      <TextField id="name" label="Full name" autoComplete="name" value={values.name}
        onChange={handleChange} onBlur={handleBlur} error={errorFor('name')} />

      <TextField id="email" label="Email" type="email" autoComplete="email" value={values.email}
        onChange={handleChange} onBlur={handleBlur} error={errorFor('email')} />

      <PasswordInput id="password" label="Password" autoComplete="new-password" value={values.password}
        onChange={handleChange} onBlur={handleBlur} error={errorFor('password')}
        describedBy="password-strength password-rules" />
      {values.password && <StrengthMeter id="password-strength" score={strength.score} label=
{strength.label} />}
      <RuleChecklist id="password-rules" rules={strength.rules} />

      <PasswordInput id="confirm" label="Confirm password" autoComplete="new-password" value=
{values.confirm}
        onChange={handleChange} onBlur={handleBlur} error={errorFor('confirm')} />

      <div className="field checkbox">
        <input id="terms" name="terms" type="checkbox" checked={values.terms} onChange={handleChange}
          onBlur={handleBlur} aria-invalid={!!errorFor('terms')}
          aria-describedby={errorFor('terms') ? 'terms-error' : undefined} />
        <label htmlFor="terms">I accept the terms and privacy policy</label>
```

```
      </div>
      {errorFor('terms') && <p id="terms-error" className="error">{errorFor('terms')}</p>}

      <button type="submit" className="submit" disabled={submitting}>
        {submitting ? 'Creating account...' : 'Create account'}
      </button>
    </form>
  )
}
```

src/components/TextField.jsx

```
export default function TextField({ id, label, error, ...inputProps }) {
  const errorId = `${id}-error`
  return (
    <div className="field">
      <label htmlFor={id}>{label}</label>
      <input
        id={id}
        name={id}
        aria-invalid={!!error}
        aria-describedby={error ? errorId : undefined}
        {...inputProps}
      />
      {error && <p id={errorId} className="error">{error}</p>}
    </div>
  )
}
```

src/components/PasswordInput.jsx

```
import { useState } from 'react'

export default function PasswordInput({ id, label, error, describedBy, ...inputProps }) {
  const [visible, setVisible] = useState(false)
  const [capsLock, setCapsLock] = useState(false)
  const errorId = `${id}-error`
  const ariaDescribedBy = [error && errorId, describedBy].filter(Boolean).join(' ') || undefined

  return (
    <div className="field">
      <label htmlFor={id}>{label}</label>
      <div className="password-wrap">
        <input
          id={id}
          name={id}
```

```
          type={visible ? 'text' : 'password'}
          aria-invalid={!!error}
          aria-describedby={ariaDescribedBy}
          onKeyUp={(e) => setCapsLock(e.getModifierState('CapsLock'))}
          {...inputProps}
        />
        <button
          type="button"
          className="toggle"
          aria-label={visible ? `Hide ${label.toLowerCase()}` : `Show ${label.toLowerCase()}`}
          aria-pressed={visible}
          onClick={() => setVisible((v) => !v)}
        >
          {visible ? 'Hide' : 'Show'}
        </button>
      </div>
      {capsLock && <p className="hint">Caps Lock is on</p>}
      {error && <p id={errorId} className="error">{error}</p>}
    </div>
  )
}
```

src/components/StrengthMeter.jsx

```
const SEGMENTS = [0, 1, 2, 3, 4]

export default function StrengthMeter({ id, score, label }) {
  return (
    <div id={id} className="strength">
      <div className="bars" aria-hidden="true">
        {SEGMENTS.map((i) => (
          <span key={i} className={i <= score ? `bar level-${score}` : 'bar'} />
        ))}
      </div>
      <p className={`strength-label text-${score}`} aria-live="polite">
        Strength: <strong>{label}</strong>
      </p>
    </div>
  )
}
```

src/components/RuleChecklist.jsx

```
export default function RuleChecklist({ id, rules }) {
  return (
    <ul id={id} className="rules">
```

```
      {rules.map((r) => (
        <li key={r.id} className={r.passed ? 'ok' : undefined}>
          <span aria-hidden="true">{r.passed ? '✓' : '○'}</span> {r.label}
          <span className="sr-only">{r.passed ? ' (met)' : ' (not met)'}</span>
        </li>
      ))}
    </ul>
  )
}
```

src/App.jsx

```
import { useState } from 'react'
import SignupForm from './components/SignupForm'

export default function App() {
  const [user, setUser] = useState(null)

  return (
    <main className="page">
      <section className="card">
        {user ? (
          <div className="success" role="status">
            <h1>Welcome, {user.name}!</h1>
            <p>Your account <strong>{user.email}</strong> has been created.</p>
            <button type="button" className="submit" onClick={() => setUser(null)}>Sign up
another</button>
          </div>
        ) : (
          <>
            <h1>Create your account</h1>
            <SignupForm onSuccess={setUser} />
          </>
        )}
      </section>
    </main>
  )
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
```

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/index.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f3f4f6; color: #1f2937; }

.page { min-height: 100vh; display: grid; place-items: center; padding: 16px; }
.card { width: 100%; max-width: 420px; background: #fff; padding: 28px; border-radius: 12px; box-shadow: 0 4px 20px rgb(0 0 0 / 0.08); }
h1 { margin: 0 0 20px; font-size: 1.5rem; }

.field { display: flex; flex-direction: column; gap: 6px; margin-bottom: 14px; }
label { font-size: 0.9rem; font-weight: 600; }
input[type='text'], input[type='email'], input[type='password'], input:not([type]) {
  width: 100%; padding: 10px 12px; font-size: 1rem; border: 1px solid #d1d5db; border-radius: 8px;
}
input:focus-visible, button:focus-visible { outline: 2px solid #2563eb; outline-offset: 2px; }
input[aria-invalid='true'] { border-color: #dc2626; }

.password-wrap { position: relative; }
.password-wrap input { padding-right: 64px; }
.toggle { position: absolute; right: 6px; top: 50%; transform: translateY(-50%); border: 0; background: none; color: #2563eb; font-weight: 600; cursor: pointer; padding: 6px; }

.error { margin: 0; color: #dc2626; font-size: 0.85rem; }
.hint { margin: 0; color: #b45309; font-size: 0.85rem; }
.server-error { background: #fef2f2; color: #b91c1c; border: 1px solid #fecaca; padding: 10px 12px; border-radius: 8px; }

.strength { margin: -6px 0 8px; }
.bars { display: grid; grid-template-columns: repeat(5, 1fr); gap: 4px; }
.bar { height: 6px; border-radius: 3px; background: #e5e7eb; transition: background 0.2s; }
.level-0 { background: #dc2626; } .text-0 strong { color: #dc2626; }
.level-1 { background: #ea580c; } .text-1 strong { color: #ea580c; }
.level-2 { background: #ca8a04; } .text-2 strong { color: #a16207; }
.level-3 { background: #16a34a; } .text-3 strong { color: #15803d; }
.level-4 { background: #15803d; } .text-4 strong { color: #166534; }
.strength-label { margin: 6px 0 0; font-size: 0.85rem; }

.rules { list-style: none; margin: 0 0 16px; padding: 0; font-size: 0.85rem; color: #6b7280; display: grid; gap: 4px; }
.rules .ok { color: #15803d; }
```

```
.checkbox { flex-direction: row; align-items: center; gap: 8px; margin-bottom: 6px; }
.checkbox label { font-weight: 400; }

.submit { width: 100%; margin-top: 12px; padding: 12px; font-size: 1rem; font-weight: 600; color: #fff;
background: #2563eb; border: 0; border-radius: 8px; cursor: pointer; }
.submit:disabled { opacity: 0.6; cursor: not-allowed; }

.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-
space: nowrap; }
```

selfcheck.mjs (run with `node selfcheck.mjs`)

```
import assert from 'node:assert/strict'
import { getStrength, validate } from './src/utils/passwordRules.js'

assert.deepEqual([getStrength('').score, getStrength('').label], [0, ''])
assert.equal(getStrength('').rules.every((r) => !r.passed), true)
assert.equal(getStrength('abc').score, 1) // lower + personal
assert.equal(getStrength('abcDEF1').score, 2) // 5 of 6 (short)
assert.equal(getStrength('Password@123').common, true)
assert.equal(getStrength('Password@123').score, 0)
assert.equal(getStrength('Tr0ub4dor&x').score, 3)
assert.equal(getStrength('Tr0ub4dor&xyz9').score, 4)
assert.equal(getStrength('Kumar@2026xyz', { name: 'Ashu Kumar', email: '' }).score, 2)
assert.equal(getStrength('Xashu#99long', { name: '', email: 'ashu@x.com' }).rules.find((r) => r.id ===
'personal').passed, false)

const ok = { name: 'Ashu', email: 'a@b.co', password: 'Tr0ub4dor&x', confirm: 'Tr0ub4dor&x', terms:
true }
assert.deepEqual(validate(ok), {})
assert.equal(validate({ ...ok, confirm: 'nope' }).confirm, 'Passwords do not match')
assert.equal(validate({ ...ok, email: 'a@b' }).email, 'Enter a valid email address')
assert.equal(validate({ ...ok, password: 'Password@123', confirm: 'Password@123' }).password, 'This
password is too common, pick another')
assert.deepEqual(Object.keys(validate({ name: '', email: '', password: '', confirm: '', terms: false
})), ['name', 'email', 'password', 'confirm', 'terms'])
console.log('selfcheck ok')
```

12. Wrap-up

Edge cases handled

- Empty password shows no false ticks; common passwords forced to score 0.
- Password containing name or email is rejected.
- Errors are derived, so confirm-mismatch can never be stale.
- Errors show after blur or submit, focus jumps to the first invalid field.

- Double submit blocked; server error kept on screen until the user edits.
- Password not trimmed; name and email trimmed; email lowercased by the API.

Trade-offs to mention

- **Rule-count scoring vs entropy (zxcvbn):** rule counting is explainable and tiny; zxcvbn catches keyboard patterns (`qwerty`) and dictionary words but is ~400 KB (lazy-load it if needed).
- **Hard-coded common list vs breached-password API:** the list is a demo; production would use a server-side check (Have I Been Pwned k-anonymity range API).
- **Derive on every render vs memo:** validation is microseconds; `useMemo` would add noise without a measurable win.
- **Client rules are UX only:** the server must enforce the same policy; the pure rules file can be shared to avoid drift.
- **Own form code vs react-hook-form + zod:** for 5 fields plain state is clearer; for a large app a library gives schema reuse and fewer re-renders.

What to add with more time

- Unit tests (Vitest) for `getStrength` / `validate` and React Testing Library tests for the flows.
- Async "email available?" check on blur, debounced with `AbortController`.
- Max length (e.g. 64-128) and Unicode normalization (`password.normalize('NFKC')`).
- i18n for labels and messages; rule labels already live in one place.
- Server-side field errors mapped to fields (e.g. 409 -> email field error).

13. Counter questions the interviewer may ask

Requirements

Q1. Why not block the submit button until the form is valid?

A disabled button gives no reason why it is disabled, and screen reader users may not find it. Keeping it enabled and showing all errors + focusing the first invalid field on click explains the problem. Disable it only while submitting.

Q2. Should a "Fair" password be allowed?

It is a product decision. We require all rules (Good or better). A softer policy is one line: change `strength.score < 3` in `validate`. NIST actually recommends length + breached-list checks over composition rules, which is worth mentioning.

Q3. Why don't you trim the password?

Spaces are valid characters and passphrases ("correct horse battery staple") are good passwords. Trimming silently changes what the user typed, so login later fails. Only name and email are trimmed.

State design

Q4. Why is `strength` not in state?

It is a pure function of `password`, `name` and `email`. Storing it means updating it in every handler and risking a stale value. Deriving it every render is cheaper than one `setState`.

Q5. Why one `values` object instead of five `useState`s?

One `handleChange` keyed by `e.target.name` handles all inputs, and `validate(values)` gets everything it needs (the personal rule needs name and email). Five states would need five handlers or a switch.

Q6. Why keep `visible` inside `PasswordInput`?

Only that input cares. Lifting it to the form would re-render the whole form for a UI toggle and couple unrelated concerns. Each password field gets its own independent toggle.

Hard parts: regex and scoring

Q7. Why not one big regex with lookaheads?

`/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s]).{8,}$/` only says pass/fail. We need per-rule status for the checklist and readable messages. Small regexes are also easier to change and test.

Q8. Is `/[^A-Za-z0-9\s]/` correct for "special character"?

It treats anything that is not a letter, digit or whitespace as special, so `é` or emoji count too. If the policy needs a fixed set, use `/[!@#$%^&*()_+\-=[\]{};':"\\|,.<>/?]/`. Mention Unicode letters with `/\p{Lu}/u` for non-ASCII uppercase.

Q9. Can a regex here cause performance problems (ReDoS)?

Our regexes are single character classes with no nested quantifiers, so they run in linear time. Danger patterns are like `(a+)+$`. Also add a max length so even a slow check has a bounded input.

Q10. How would you use zxcvbn instead?

`const { score } = zxcvbn(password, [name, email])` returns 0-4 and feedback. Since it is ~400 KB, load it with `import('zxcvbn')` when the password field gets focus, and show the rule checklist until it is ready.

Q11. What if the common-password list has 100,000 entries?

Do not ship it to the client. Check on the server or use the HIBP range API: send only the first 5 chars of the SHA-1 hash, compare suffixes locally. Debounce the call (~400 ms) and cancel stale requests with `AbortController`.

Implementation steps

Q12. Why show errors on blur and not on change?

On change, "Enter a valid email" flashes at the first character. On blur the user has finished their attempt. A common refinement: after a field shows an error, re-validate it on change so the error disappears as soon as it is fixed (our derived errors already do that).

Q13. How do you focus the first invalid field?

`validate` builds errors in DOM order, so `Object.keys(errors)[0]` is the first one. `form.elements[name]` finds the input by its `name` attribute; no refs or ids needed.

Q14. What if the API is slow or the user double-clicks?

The `status === 'submitting'` guard returns early and the button is disabled with a loading label. For a very slow API add a timeout with `AbortController` and a retry message. The server should also be idempotent on email (unique constraint).

Q15. How do you show a server error on the right field?

Map the API response to fields: on 409 set `serverErrors.email`, merge it into `errorFor('email')`, and clear it when the email changes. Generic errors stay in the top `role="alert"` banner.

Q16. Is it safe to show the password in plain text with the toggle?

It is the user's choice and reduces typos (some sites drop the confirm field because of it). Keep `type="button"` so it does not submit, never log the value, and consider hiding it again on blur for shared screens.

Accessibility

Q17. How does a screen reader user know the strength?

The text label sits in an `aria-live="polite"` region, so changes are announced after typing pauses. The input's `aria-describedby` points to the meter and checklist, and each rule has hidden "(met)/(not met)" text. Bars are `aria-hidden` because they are only decoration.

Q18. Why `autoComplete="new-password"`?

It tells password managers this is a new password, so they offer to generate a strong one and don't autofill the old one. The email field uses `autoComplete="email"`.

Performance and scale

Q19. Every keystroke re-runs all rules and re-renders the form. Is that a problem?

Six regex tests on a string under 128 chars take microseconds, far below a frame. If an expensive check (zxcvbn, API) is added, use `useDeferredValue(password)` or debounce only that check, keeping the checklist instant.

Q20. What if this becomes a 30-field form?

Every keystroke re-rendering 30 fields could start to cost. Options: `React.memo` on fields with stable handlers, uncontrolled inputs with `react-hook-form`, and a schema (zod) shared with the backend.

Trade-offs and "what if"

Q21. What if the backend has a different policy?

Policies drift when they are written twice. Fetch the policy (min length, required classes) from a config endpoint, or publish `passwordRules.js` as a shared package used by both client and Node server.

Q22. Is client-side validation enough for security?

No. It is only for UX. Anyone can call the API directly, so the server must enforce the same rules, hash with bcrypt/argon2, rate-limit signups and send the password only over HTTPS.

Q23. What if the requirement says "max 64 characters"?

Add a rule `{ id: 'max', test: (p) => p.length <= 64 }` and `maxLength={64}` on the input. Note bcrypt ignores bytes after 72, which is a common reason for a max length.